Лабораторна робота №4

Тема: Розширені можливості Node.js-додатків: логування, завантаження файлів, моніторинг продуктивності

Мета роботи

Ознайомитися з розширеними можливостями серверних застосунків на базі Node.js, а саме:

- реалізацією логування запитів і подій
- організацією завантаження файлів на сервер
- моніторингом продуктивності застосунку

Теоретичні відомості

1. Логування в Node.js

Логування — це процес фіксації подій у системі (запити, помилки, дії користувача).
Популярні інструменти:

- Morgan — логування HTTP-запитів
- Winston — гнучке логування

1.1 Що таке логування в Node.js

Логування — це запис інформації про те, що відбувається у програмі.

Логування потрібно для:

1. **Пошуку помилок**
   → бачиш, де саме впав код
2. **Контроль користувачів**
   → хто і коли робив запити
3. **Аналізу продуктивності**
   → які запити повільні
4. **Безпеки**
   → підозріла активність

У нормальному логуванні є рівні:

- `info` — звичайні події
- `warn` — щось підозріле
- `error` — помилки
- `debug` — деталі для розробника

Приклад:

```
logger.info("User logged in");
 logger.error("Database connection failed");
```

1.2 Логування HTTP-запитів (Morgan)

Morgan автоматично записує всі запити до сервера.

Підключення:

```
const morgan = require('morgan');
app.use(morgan('dev'));
```

Що бачимо:

```
GET / 200 5ms
POST /upload 201 12ms
```

Тобто:

- метод (GET/POST)
- шлях
- статус
- час

1.3 Професійне логування (Winston)

Winston — це вже «серйозний рівень».

Налаштування:

```
const winston = require('winston');

const logger = winston.createLogger({
  level: 'info',
  format: winston.format.json(),
  transports: [
    new winston.transports.File({ filename: 'combined.log' })
  ]
});
```

Використання:

```
logger.info('Server started');
logger.error('Something went wrong');
```

Як виглядає лог:

```
{
  "level": "info",
  "message": "Server started",
  "timestamp": "2026-04-29T10:00:00Z"
}
```

1.4 Логування помилок

```
app.use((err, req, res, next) => {
  logger.error(err.message);
  res.status(500).json({ error: "Server error" });
});
```

1.5 Логування часу виконання

```
app.use((req, res, next) => {
  const start = Date.now();

  res.on('finish', () => {
    const duration = Date.now() - start;
    logger.info(`${req.method} ${req.url} - ${duration}ms`);
  });

  next();
});
```

1.6 Як це все працює разом

У реальному проєкті:

- Morgan → логує всі HTTP-запити
- Winston → записує події та помилки у файл
- middleware → вимірює продуктивність

1.7 Типова структура логів

У тебе можуть бути:

- `access.log` → всі запити
- `error.log` → помилки
- `combined.log` → усе разом

Для запам'ятовування

Логування = запис того, що відбувається в програмі

Хороше логування = структуроване + з рівнями + у файл

2. Завантаження файлів

Для обробки файлів у Node.js використовують middleware:

- Multer

Коли користувач додає фото, надсилає PDF , завантажує документ через форму браузер відправляє файл на сервер. Пторібно **прийняти цей файл, обробити і зберегти**.

Node.js (і Express.js) **не вміє сам нормально обробляти файли**.

Коли файл надсилається, він приходить у спеціальному форматі: **`multipart/form-data`**

Це не JSON і не простий текст — це складна структура.

Multer — це middleware, який:

- читає файл із запиту
- витягує його
- зберігає на сервер
- додає інформацію про файл у req

Middleware — це функція, яка: перехоплює запит ДО того, як він дійде до твого коду

У випадку Multer:

1. приходить запит з файлом
2. Multer його обробляє
3. передає результат у твій handler

2.1. HTML-форма (або Postman)

```
<form action="/upload" method="POST" enctype="multipart/form-data">
  <input type="file" name="file" />
```

```
  <button type="submit">Upload</button>
</form>
```

Важливо:

- `enctype="multipart/form-data"` — без цього файл не відправиться

2.2 Підключення Multer

```
const multer = require('multer');
```

2.3. Налаштування збереження

```
const upload = multer({
  dest: 'uploads/'
});
```

Що це означає:

- всі файли будуть збережені в папку `uploads/`
- ім'я файлу буде згенероване автоматично

2.4. Endpoint для завантаження

```
app.post('/upload', upload.single('file'), (req, res) => {
  res.send('Файл завантажено');
});
```

Важливо:

```
upload.single('file')
```

- `single` → означає **один файл**
- `'file'` → ім'я поля з HTML (`name="file"`)

Що з'являється в req

Після Multer:

```
req.file
```

Містить:

```
{
  fieldname: 'file',
  originalname: 'photo.jpg',
  encoding: '7bit',
  mimetype: 'image/jpeg',
  destination: 'uploads/',
  filename: 'abc123.jpg',
  path: 'uploads/abc123.jpg',
  size: 12345
}
```

Завантаження кількох файлів

```
app.post('/upload-multiple', upload.array('files', 5), (req, res) => {
  res.send('Файли завантажено');
});
```

- `'files'` — ім'я поля
- `5` — максимум файлів

```
req.files
```

2.5 Кастомне збереження (важливо для лабораторної)

Щоб контролювати ім'я файлу:

```
const storage = multer.diskStorage({
  destination: (req, file, cb) => {
    cb(null, 'uploads/');
  },
  filename: (req, file, cb) => {
    cb(null, Date.now() + '-' + file.originalname);
  }
});

const upload = multer({ storage });
```

2.6 Валідація файлів (дуже важливо)

1. Тип файлу

```
const fileFilter = (req, file, cb) => {
  if (file.mimetype === 'image/jpeg' || file.mimetype === 'image/png') {
    cb(null, true);
  } else {
    cb(new Error('Неправильний формат'), false);
  }
};
```

2. Розмір файлу

```
const upload = multer({
  dest: 'uploads/',
  limits: { fileSize: 1024 * 1024 } // 1MB
});
```

Повний приклад

```
const express = require('express');
const multer = require('multer');

const app = express();

const storage = multer.diskStorage({
  destination: (req, file, cb) => {
    cb(null, 'uploads/');
  },
  filename: (req, file, cb) => {
    cb(null, Date.now() + '-' + file.originalname);
  }
});

const upload = multer({ storage });

app.post('/upload', upload.single('file'), (req, res) => {
  res.json({
    message: 'Файл завантажено',
    file: req.file
  });
});

app.listen(3000);
```

Послідовність

1. Користувач відправляє файл
2. Запит приходить на сервер
3. Multer:
    - читає файл

    - зберігає його
    - додає у `req.file`
4. Твій код повертає відповідь

Для запам'ятовування

Multer = **інструмент**, який дозволяє серверу «**розуміти**» **файли**

Без нього: ❌
З ним: ✔ можна приймати, зберігати і перевіряти файли

3. Моніторинг продуктивності

Дає змогу оцінити:

- час відповіді сервера
- використання пам'яті
- навантаження CPU

Інструменти:

- вбудований модуль `process`
- бібліотеки типу PM2

Що таке моніторинг продуктивності

Моніторинг продуктивності — це процес спостереження за тим, **як працює сервер або додаток** у реальному часі.

Його мета:

- виявити "вузькі місця" (де система гальмує)
- оцінити стабільність роботи
- запобігти збоям

Основні показники

1. Час відповіді сервера (Response Time)

Це час, за який сервер обробляє запит і повертає відповідь клієнту(наскільки швидко сервер реагує)

Формула:

`Час відповіді = час отримання запиту → час відправки відповіді`

Чому це важливо:

- великий час відповіді = поганий UX
- може означати перевантаження або неефективний код

Приклад у Node.js:

```
app.use((req, res, next) => {
  const start = Date.now();

  res.on("finish", () => {
```

```
    const duration = Date.now() - start;
    console.log(`Запит оброблено за ${duration} мс`);
  });

  next();
});
```

---

2. Використання пам’яті (Memory Usage)

Показує, скільки оперативної пам’яті використовує програма.

Важливі метрики:

- `heapUsed` — використана пам’ять
- `heapTotal` — виділена пам’ять
- `rss` — загальна пам’ять процесу

Чому це важливо:

- допомагає знайти витоки пам’яті (memory leaks)
- перевищення → краш програми

Приклад:

```
const memory = process.memoryUsage();

console.log(memory);
/*
{
  rss: 23482368,
  heapTotal: 4907008,
  heapUsed: 2905912
}
*/
```

3. Навантаження CPU (CPU Load)

Показує, наскільки процесор зайнятий виконанням задач.

Чому це важливо:

- високий CPU → повільна робота
- може означати:
  - складні обчислення
  - нескінченні цикли
  - погану оптимізацію

Приклад:

```
const usage = process.cpuUsage();

console.log(usage);
/*
{
  user: 123456,
  system: 78910
}
*/
```

Інструменти для моніторингу

1. Вбудований модуль process (Node.js)

Це стандартний модуль, який дає доступ до інформації про поточний процес.

Можливості:

- отримання пам'яті → `process.memoryUsage()`
- CPU → `process.cpuUsage()`
- uptime → `process.uptime()`

Приклад комплексного моніторингу:

```
setInterval(() => {
  const memory = process.memoryUsage();
  const cpu = process.cpuUsage();

  console.log("=== Моніторинг ===");
  console.log("Пам'ять:", memory.heapUsed);
  console.log("CPU:", cpu.user);
}, 5000);
```

Плюси:

- не потребує встановлення
- простий у використанні

Мінуси:

- немає графіків
- тільки базова інформація

2. PM2

PM2 — це популярний менеджер процесів для Node.js, який також має вбудований моніторинг.

Що він дає:

- запуск і контроль додатків
- автоматичний перезапуск
- моніторинг у реальному часі

Встановлення:

```
npm install -g pm2
```

Запуск додатку:

```
pm2 start app.js
```

Моніторинг:

```
pm2 monit
```

Відкривається інтерактивна панель з:

- CPU
- пам'яттю
- статусом процесів

Додаткові можливості PM2

```
pm2 list        # список процесів
pm2 logs        # перегляд логів
pm2 restart app # перезапуск
```

Порівняння інструментів

Інструмент	Плюси	Мінуси
process	простий, вбудований	мало функцій
PM2	зручний, графічний моніторинг	потребує встановлення

Висновок

Моніторинг продуктивності дозволяє:

- контролювати стан сервера
- виявляти проблеми (затримки, витоки пам'яті)
- оптимізувати роботу системи

Основні метрики:

- час відповіді
- пам'ять
- CPU

Основні інструменти:

- `process` — для базового аналізу
- PM2 — для повноцінного моніторингу

Кінець форми

Хід роботи

Завдання 1: Створення базового сервера

1. Ініціалізуйте проєкт:

```
npm init -y
```

2. Встановіть залежності:

```
npm install express morgan winston multer
```

3. Створіть файл `app.js`:

```
const express = require('express');
const app = express();

app.get('/', (req, res) => {
    res.send('Server is running');
});

app.listen(3000, () => {
```

```
    console.log('Server started on port 3000');
});
```

Завдання 2: Логування запитів

Додайте Morgan:

```
const morgan = require('morgan');
app.use(morgan('combined'));
```

Додайте Winston:

```
const winston = require('winston');

const logger = winston.createLogger({
    level: 'info',
    transports: [
        new winston.transports.File({ filename: 'app.log' })
    ]
});

logger.info('Server started');
```

Завдання 3: Завантаження файлів

```
const multer = require('multer');

const upload = multer({ dest: 'uploads/' });

app.post('/upload', upload.single('file'), (req, res) => {
    res.send('File uploaded successfully');
});
```

Завдання 4: Моніторинг продуктивності

```
app.get('/status', (req, res) => {
    const memoryUsage = process.memoryUsage();
    const uptime = process.uptime();

    res.json({
        uptime,
        memoryUsage
    });
});
```

Завдання 5 (додаткове): Логування продуктивності

```
app.use((req, res, next) => {
    const start = Date.now();

    res.on('finish', () => {
        const duration = Date.now() - start;
        console.log(`${req.method} ${req.url} - ${duration}ms`);
    });

    next();
});
```

Практичні завдання

1. Ініціалізація проєкту

Запустити серверний застосунок на базі Node.js із використанням Express.
Переконайтеся, що сервер відповідає на GET-запит `/`.

2. Логування HTTP-запитів

Підключіть Morgan та налаштуйте логування всіх HTTP-запитів у консоль.

3. Файлове логування подій

Інтегруйте Winston і реалізуйте:

- запис логів у файл `app.log`
- логування рівнів: `info`, `error`

4. Обробка помилок

Реалізуйте middleware для обробки помилок:

- логування помилок через Winston
- повернення користувачу JSON-відповіді з кодом помилки

5. Завантаження одного файлу

Реалізуйте endpoint `/upload`, який дозволяє завантажити один файл за допомогою Multer.

6. Завантаження кількох файлів

Розширте попереднє завдання:

- підтримка завантаження кількох файлів
- збереження їх у папку `/uploads`

7. Валідація файлів

Додайте перевірки:

- дозволені типи файлів (наприклад: jpg, png, pdf)
- обмеження розміру файлу

8. Моніторинг стану сервера

Створіть endpoint `/status`, який повертає:

- uptime сервера
- використання пам’яті (`process.memoryUsage()`)

9. Вимірювання часу відповіді

Реалізуйте middleware, який:

- вимірює час обробки кожного запиту
- записує результат у лог

10. Інтеграція менеджера процесів

Запустіть застосунок через PM2 та:

- перевірте автоматичний рестарт
- перегляньте логи через PM2

Додаткове (за бажанням / бонус)

- Реалізувати ротацію логів
- Додати API для перегляду логів
- Побудувати просту панель моніторингу

Контрольні запитання

1. Що таке middleware у Node.js?
2. Яка різниця між Morgan і Winston?
3. Як працює Multer?
4. Які метрики продуктивності важливі для серверів?
5. Що таке uptime?